第三章 SQL 注入漏洞与防护——实验分析报告

学号：__0113_____姓名：___汪雪怡_____

一、作业目标
解释 SQL 注入的形成条件与根因。
在 DVWA 中观察和复现基础 SQL 注入现象。
区分普通 SQL 注入与盲注在证据层的差异。
比较 Low 与 Medium 控制思路的差异。
从参数化查询、错误处理和最小权限角度解释防护链条。

二、作业主线
本章核心主线：输入 → 拼接 → 解析 → 执行 → 回显 / 行为差异 → 结构分离防护 SQL 注入的核心本质：SQL 注入的根因是查询结构被污染，而非个别字符本身危险。

三、环境与授权边界
1. 实验环境
DVWA、localhost / 127.0.0.1、本地浏览器、phpMyAdmin
2. 操作规范
所有实验仅在授权本地环境进行，未使用公网未授权目标，截图敏感信息全部脱敏，严格遵守实验安全红线。

任务 1：数据库与查询条件基础说明
具体业务场景：一个简单的“用户信息查询”页面，用户在一个输入框中输入“用户 ID”，点击“查询”按钮，页面下方显示该用户的姓名和注册邮箱。

1. 页面中的输入为什么会与数据库查询有关

因为这个页面需要展示动态内容——不同用户输入不同 ID，页面要返回对应的不同用户信息。这些用户信息（如姓名、邮箱）不是写死在页面代码里的，而是存储在后台的数据库中。所以页面必须根据用户输入的 ID 去数据库里“查找”，才能拿到正确的信息并显示出来。

2. 最小业务链路

用户在输入框键入 ID → 前端将 ID 作为参数发送给后端 → 后端脚本接收 ID 并拼接到 SQL 查询语句中 → 数据库执行 SQL 查询 → 数据库将查询结果返回给后端 → 后端将结果嵌入 HTML 页面返回给浏览器 → 用户看到查询结果

简单概括就是：输入条件 → 数据库查询 → 返回结果。

3. 用户输入是如何进入查询条件的

以最常见的 PHP + MySQL 为例：前端通过 GET 或 POST 方法将用户输入的 ID 传给后端（例如 http://example.com/user.php?id=1）。后端脚本里用一个变量（如 $id = $_GET['id']）接收这个值，然后直接把这个变量拼接到 SQL 字符串中：

$sql = "SELECT name, email FROM users WHERE user_id = " . $id;

这样用户输入的“1”就成了查询条件的一部分。

4.为什么这为 SQL 注入创造了可能性

因为用户输入被直接、无过滤地拼接到 SQL 语句的结构中。正常输入 1 对应 WHERE user_id = 1。但攻击者可输入 1 OR 1=1，拼接后的 SQL 变为：……

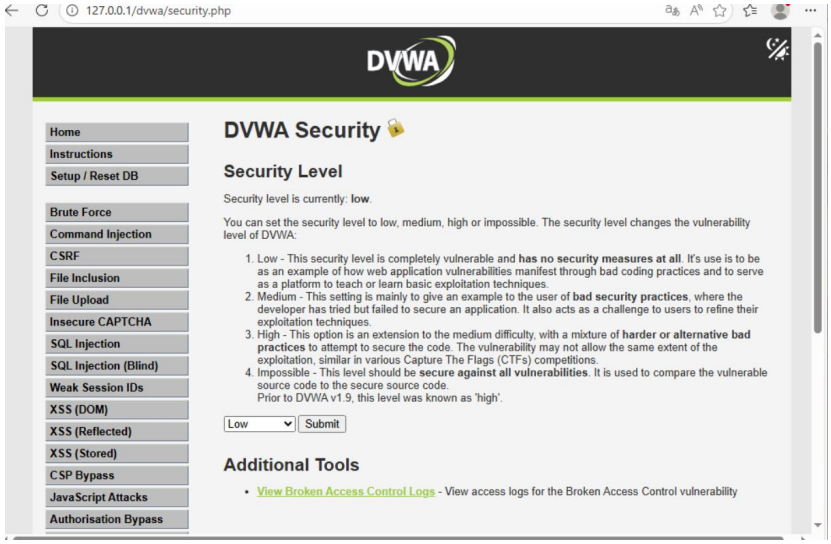
SELECT name, email FROM users WHERE user_id = 1 OR 1=1

由于 1=1 永远为真，查询返回所有用户数据。更危险的如 1; DROP TABLE users; --。

注入的根本原因不是某些字符“危险”，而是输入内容被当作 SQL 代码的一部分来解析，而不是仅仅作为数据。

任务 2：DVWA SQL Injection Low 现象记录

实验环境：DVWA Security = Low，模块 = SQL Injection（SQL 注入漏洞模块）

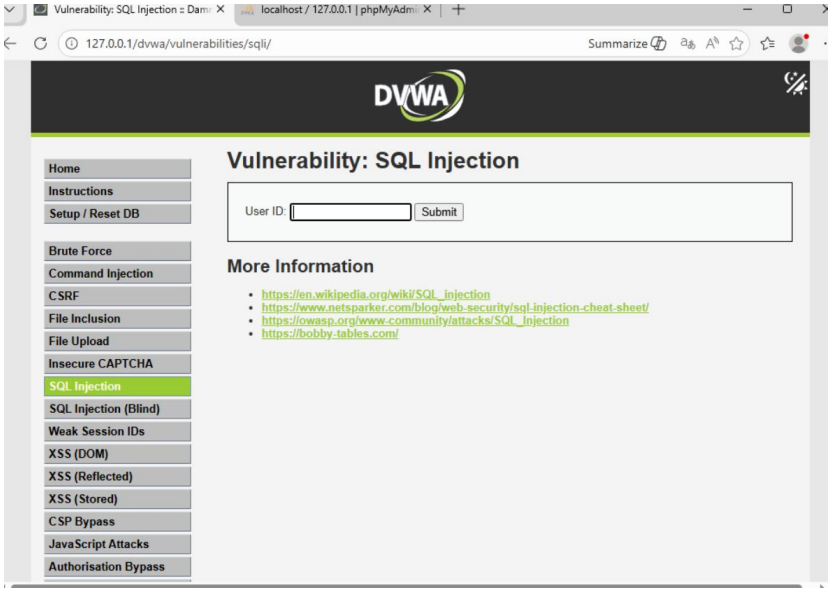


（DVWA Security 设置为 Low 级别界面）

分析：证明实验在 Low 安全级别下运行，该级别后端代码对用户输入未做任何过滤和检查，便于观察 SQL 注入的完整发生过程。

1. 实验页面/实验目标

本实验针对 DVWA 平台中的 SQL Injection（SQL 注入） 模块。该页面提供了一个文本输入框，要求用户输入一个 User ID，点击 Submit 按钮后，页面会显示该 ID 对应的用户 First name 和 Surname。实验目标是观察正常输入与恶意输入在返回结果上的差异，证明输入如何改变查询结构。

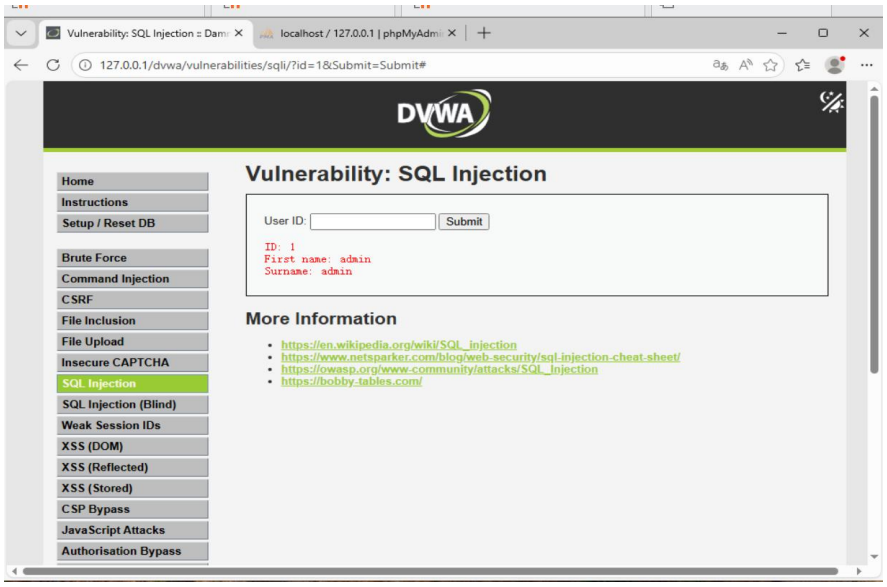


（SQL Injection 页面初始状态，包含输入框和 Submit 按钮）

2. 普通输入与异常输入的对比

2.1 普通输入（基准状态）

输入一个正常的 User ID 参数（1），点击提交。



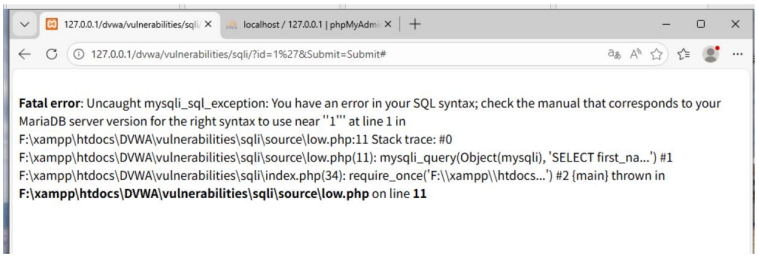
（输入 1 后的返回结果：显示 ID: 1、First name: admin、Surname: admin）

页面正常返回 ID=1 的单条记录，符合业务预期——输入某个 ID 只返回该 ID 对应的唯一

记录。

2.2 异常输入——注入点探测（单引号）

输入异常的测试参数（ 1'），点击提交。

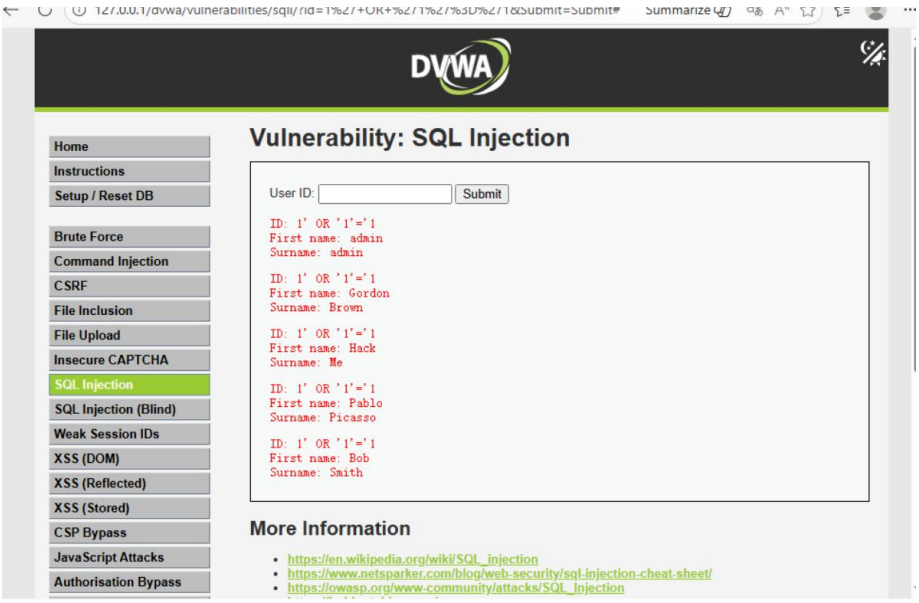


（输入 1' 后的返回结果：MySQL 语法错误信息）

分析：单引号导致数据库返回了 SQL 语法错误信息。这直接证明：用户输入未经任何过滤和转义，被完整地拼接进了 SQL 查询语句中，并且拼接后破坏了原有的 SQL 语法结构。在正常的业务逻辑中，无论输入什么字符串，页面都不应直接报出数据库原始错误。这个现象是 SQL 注入点存在 的直接证据。

2.3 异常输入——利用注入改变业务逻辑

输入构造的 payload（ 1' OR '1'='1），利用单引号闭合原查询条件并用 OR 拼接永真条件，点击提交。



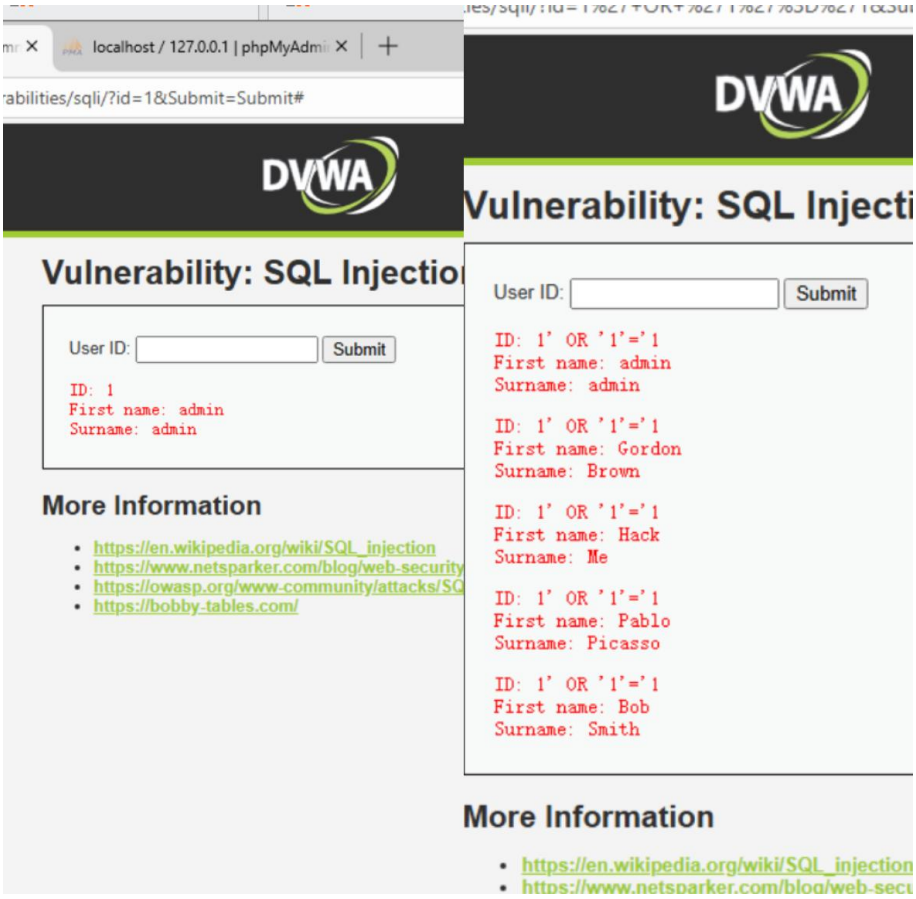
（输入 1' OR '1'='1 后的返回结果：返回了 Users 表中所有用户的多条记录）

分析：返回了 Users 表中所有用户的 First name 和 Surname（例如 admin、Gordon Brown、Hack Me、Pablo Picasso、Bob Smith 等）。这个结果与普通输入（只返回一条记录）存在显著差异。普通查询条件下，改变 ID 参数值（如 1、2、3）最多只会返回一条不同的记录，

不可能一次性返回所有用户的多条记录。

3. 返回结果的差异总结

输入内容	页面返回结果	业务预期
1（正常 ID）	仅返回 ID=1 的一条用户记录	✅ 符合
2（正常 ID）	仅返回 ID=2 的一条用户记录	✅ 符合
1'（单引号）	数据库语法错误，暴露错误信息	❌ 不符合，不应报错
1' OR '1'='1	返回所有用户的多条记录	❌ 不符合，远超业务预期



（正常输入 1 与恶意输入 1' OR '1'='1 的返回结果并排对比）

分析：并排对比可见：普通输入返回单条管理员信息，恶意输入返回整个用户表。正常应用程序不会设计“输入 ID=1 返回所有用户”的逻辑，这种差异只能解释为：用户输入的 payload 被当作 SQL 代码的一部分执行了。

4. 为什么这种差异不像“正常查询变化”

在正常业务逻辑下，输入 1 得用户 A，输入 2 得用户 B——变化的是筛选条件的内容，查询

的结构始终不变（始终是 WHERE user_id = 用户输入的值）。

而在注入 payload 1' OR '1'='1 的场景下，变化的是查询结构本身。拼接后的 SQL 从 SELECT ... WHERE user_id = '1' 变成了

SELECT ... WHERE user_id = '1' OR '1'='1'，原始 = 条件被替换为永真条件 OR '1'='1'，导致返回全表。

关键判定：这种差异不是因为筛选条件内容的变化，而是因为查询结构被改变。这正是 SQL 注入的根因——输入进入了不该进入的查询结构。

任务 3：机制分析与差异比较

1. 输入如何进入查询结构

以 DVWA Low 级别的核心代码为例（通过 View Source 查看）：

vulnerabilities/sqli/source/low.php

```php
<?php

if( isset( $_REQUEST[ 'Submit' ] ) ) {
        // Get input
        $id = $_REQUEST[ 'id' ];

        switch ($_DVWA[ 'SQLI_DB' ]) {
                case MYSQL:
                        // Check database
                        $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
                        $result = mysqli_query($GLOBALS["___mysqli_ston"], $query ) or die( '<pre>' . ((is_object($GLOBALS["___mysqli_s

                        // Get results
                        while( $row = mysqli_fetch_assoc( $result ) ) {
                                // Get values
                                $first = $row["first_name"];
                                $last  = $row["last_name"];

                                // Feedback for end user
                                echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
                        }

                        mysqli_close($GLOBALS["___mysqli_ston"]);
                        break;
                case SQLITE:
                        global $sqlite_db_connection;

                        #$sqlite_db_connection = new SQLite3($_DVWA['SQLITE_DB']);
                        #$sqlite_db_connection->enableExceptions(true);

                        $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
                        #print $query;
                        try {
                                $results = $sqlite_db_connection->query($query);
                        } catch (Exception $e) {
                                echo 'Caught exception: ' . $e->getMessage();
                                exit();
                        }

                        if ($results) {
```

（Low 级别 View Source 完整截图）

完整链条：用户输入 → $_REQUEST['id'] 接收 → 直接拼接进 SQL 字符串 → mysql_query() 执行 → 若失败，or die 直接输出数据库原始错误 → 若成功，while 循环输出查询结果。

概括为：输入 → 拼接 → 解析 → 执行 → 回显或行为差异链条。

2. 为什么根因是“结构被污染”而不是“字符本身危险”

危险字符（'、;、-- 等）在正常数据中也可能出现（如姓名 I'Wxy）。若使用参数化查询，这些字符只是普通数据。

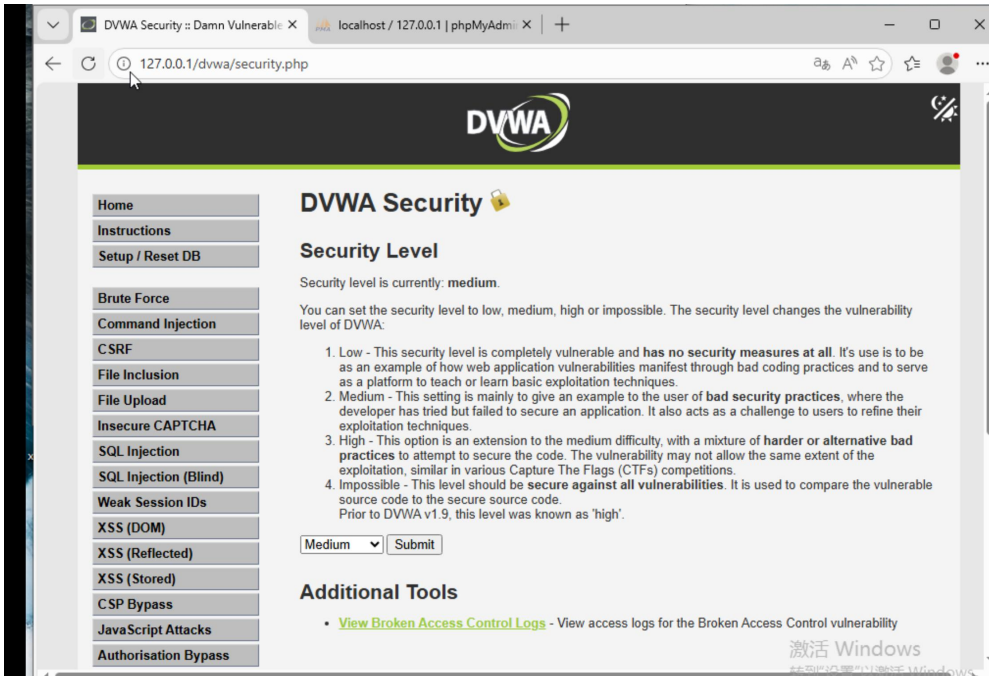
注入发生的真正原因：程序将用户输入直接拼接到 SQL 语句的语法结构中，使输入内容被数据库解析器当作 SQL 代码的一部分执行。

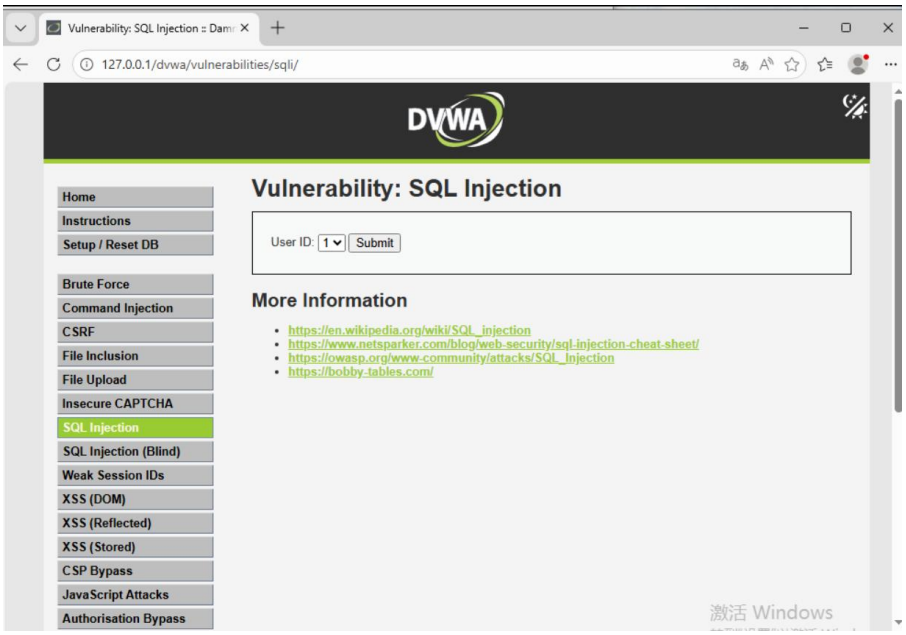
例如输入 I'Wxy 作为姓名存储时无害，但输入 1' OR '1'='1 在拼接情况下变成 WHERE user_id = '1' OR '1'='1'，其中 OR '1'='1' 成了查询结构的一部分。

结论：根因是输入进入了不该进入的查询结构，而不是字符本身“危险”。

3. Low 与 Medium 的差异

对比维度	Low 级别	Medium 级别
防护措施	无任何过滤或转义	使用 mysqli_real_escape_string() 转义特殊字符；前端输入框改为下拉选择框
输入方式	自由文本输入框	下拉菜单（固定几个 ID 值）
后端处理思路	直接拼接	转义后拼接
是否存在注入	存在（字符型/数字型均可）	仍然存在（数字型注入可绕过转义）
本质缺陷	未分离查询结构与数据	仍采用“拼接”模式，仅试图过滤危险字符





（Medium 界面）

Medium：

Low：

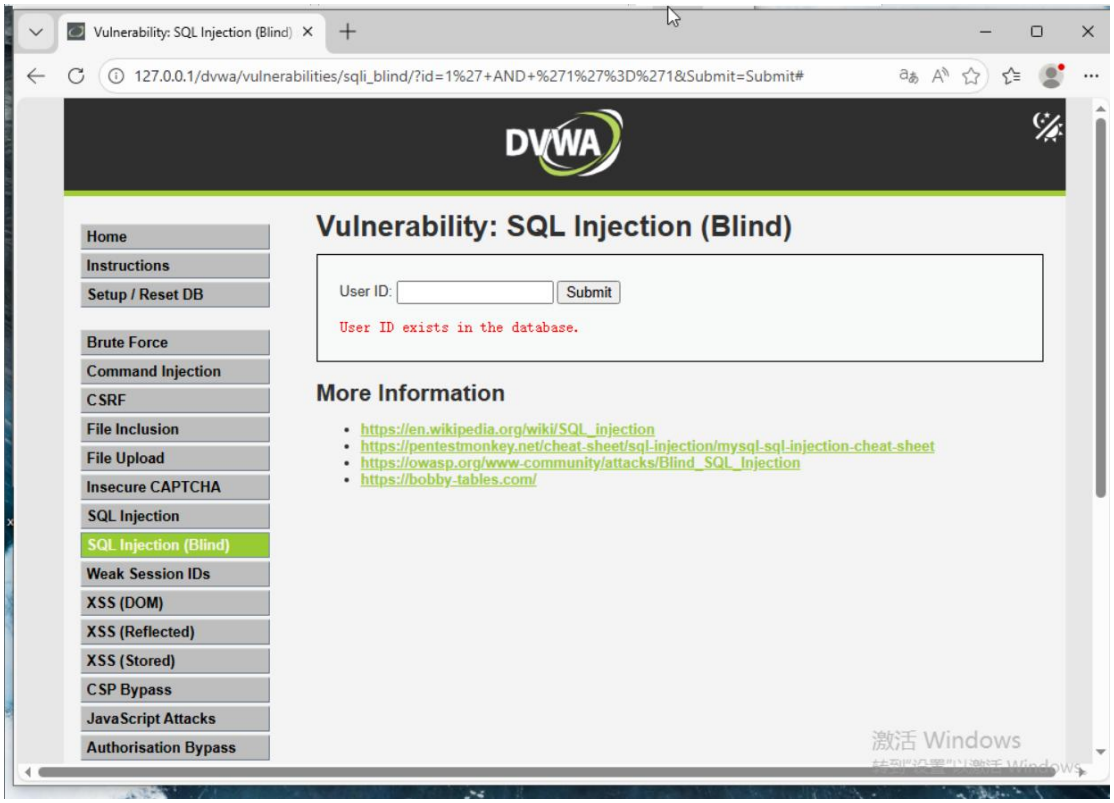
（Low 与 Medium 源码对比：Low 无转义，Medium 有 mysqli_real_escape_string）

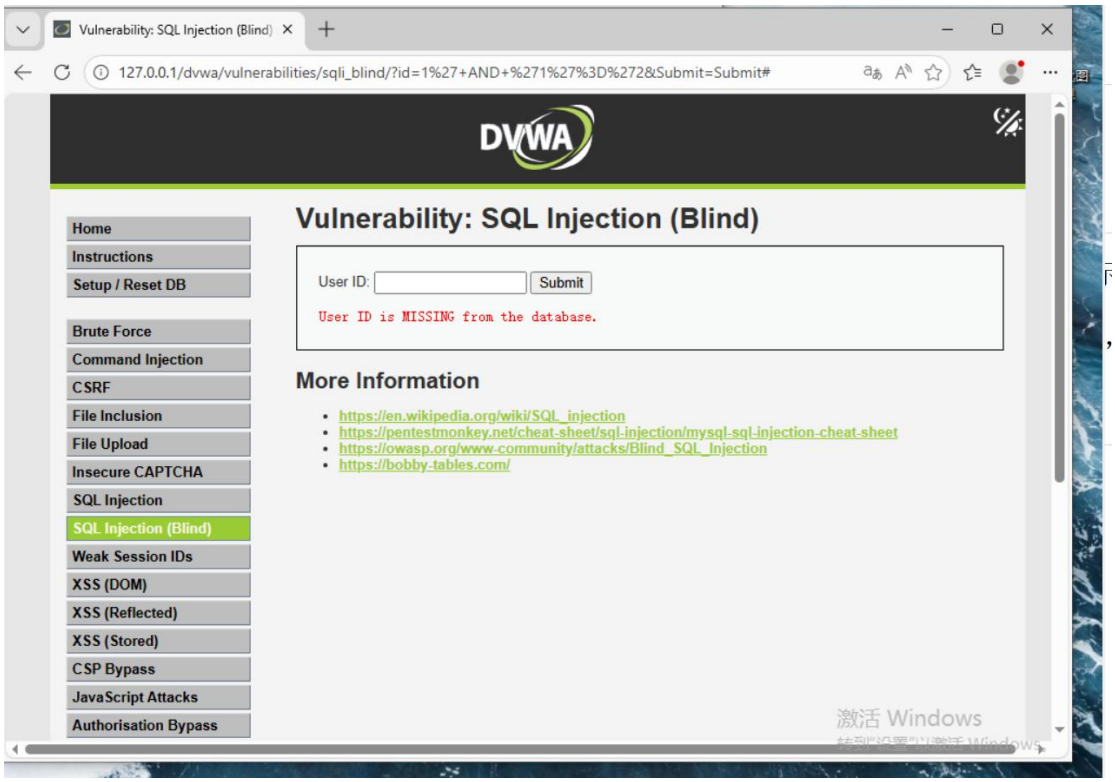
分析：Low 完全不做任何防护，输入直接进入查询结构。Medium 做了一层“字符过滤”，但没有改变“拼接”模式，查询结构依然可以被数字型 payload（如 1 OR 1=1#）污染——因为数字型注入不需要使用单引号，转义函数对此无效。真正的根因控制应该是参数化查询（结构分离），而不是字符过滤。

4. 普通 SQL 注入与 Blind SQL Injection（盲注）的差异

类型	页面回显情况	证据形式	例子

类型	页面回显情况	证据形式	例子
普通注入	查询结果直接显示在页面上，或数据库错误信息暴露	可见数据（如用户名、密码）、错误提示	输入 1' OR '1'='1 返回所有用户
盲注	页面不显示查询结果，只显示“存在/不存在”、“真/假”状态，或响应时间差异	行为差异（如两个不同条件返回不同提示）、时间延迟	输入 1' AND '1'='1 显示“存在”，输入 1' AND '1'='2 显示“不存在”





（Blind SQL Injection 页面：正常输入显示“User ID exists”，异常条件显示“User ID is missing”）

分析：普通注入像“直接看到抽屉里的东西”，盲注像“只能通过敲击墙壁听回音来判断”。但两者利用的是同一个漏洞：输入被拼接到查询结构中，改变了查询逻辑。

5. 为什么盲注和普通注入根因相同，但证据形式不同

根因相同：都是后端将用户输入直接拼接到 SQL 语句中，导致查询结构可以被攻击者控制。无论是普通注入还是盲注，底层的漏洞成因完全一致。

证据形式不同：这是因为应用程序对查询结果的处理方式不同：

普通注入的页面会直接输出数据库返回的数据集（如用户表内容）或将数据库错误信息打印到浏览器。

盲注的页面只根据数据集是否为空返回不同的布尔状态（如“存在/不存在”），或者根据查询条件执行时间差异（如 SLEEP(5)）来表现，不输出具体数据。

所以，攻击者在盲注场景下需要更复杂的技巧（如逐字符猜测、时间盲注）来提取信息，但漏洞的本质没有变化。

强制判断题

1. SQL 注入的根因更接近

选择 **B.** 输入进入了不该进入的查询结构

理由：危险字符只是攻击利用的工具。如果输入没有进入查询结构（例如使用了参数化查询），即使输入包含 ' OR 1=1，也只会被当作普通字符串处理，不会造成危害。

2. Blind SQL Injection 和普通 SQL 注入的最大差异更接近

选择 **B**. 证据形式不同

理由：两者根因完全相同——都是查询结构被污染。唯一区别是应用程序对查询结果的呈现方式不同：一个直接回显数据，一个只回显状态。不能说“根因不同”，那是错误理解。

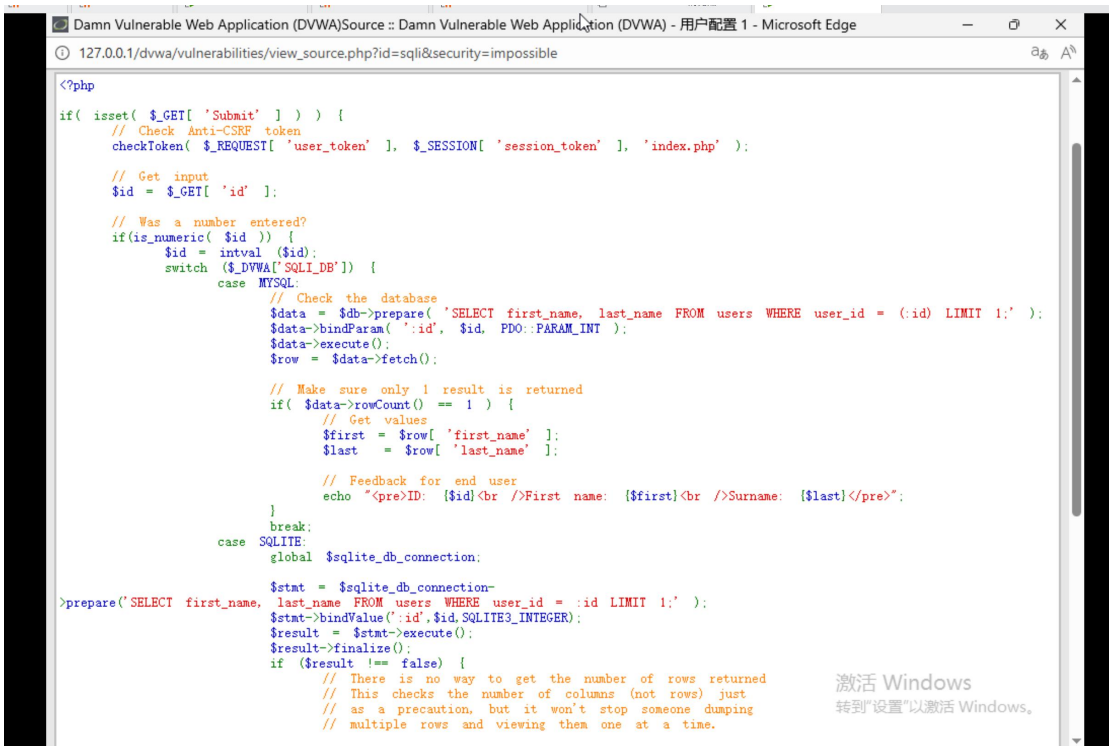
对比总结表：

对比项	普通 SQL 注入	Blind SQL Injection	Low vs Medium
根因	结构被污染	结构被污染	Low: 无防护；Medium: 字符过滤（仍为拼接模式）
证据形式	数据/错误回显	行为差异/时间延迟	-
控制思路	-	-	Low: 不控制；Medium: 减少暴露面+过滤字符（非根因控制）
能否防御	参数化查询	参数化查询	参数化查询（结构分离）

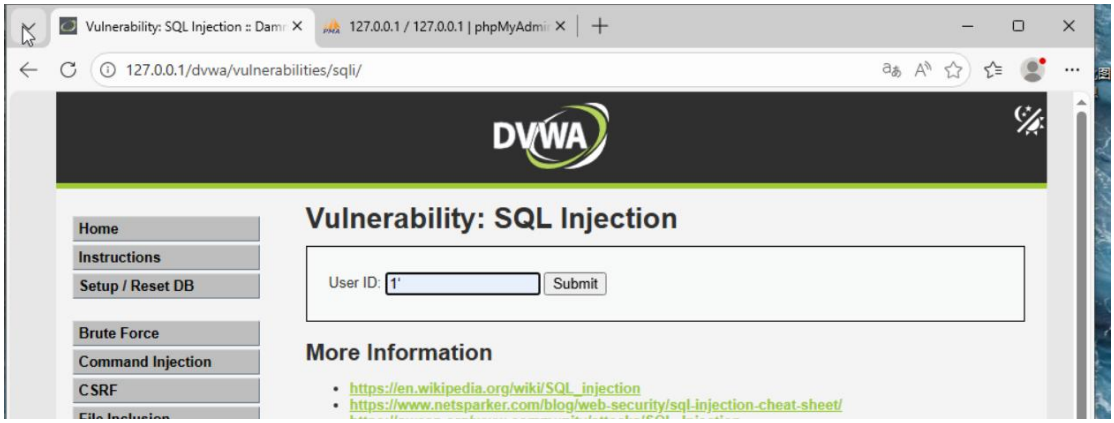
任务 4：SQL 注入防护清单与解释

措施	解决什么问题	控制层	不能替代什么
参数化查询（预编译）	将 SQL 语句的结构与参数数据分离，确保用户输入永远只能作为数据值被传递，而无法改变查询的语法结构。	根因控制	不能替代输入校验（如业务合法性检查）；对某些存储过程内的动态拼接无效；不能防御其他类型的注入（如命令注入）。
输入校验（白名单）	限制用户输入的格式、类型、长度或取值范围（例如 ID 只能是数字，用户名只能包含字母），减少攻击者可利用的输入面。	暴露面控制	不能替代参数化查询（黑名单校验极易被绕过，白名单也可能存在逻辑漏洞）；对于复杂输入（如 JSON、富文本）难以完全覆盖。
错误处理（自定义错误页）	禁止将数据库原始错误信息（如 SQL 语法错误、表名、字段名）直接返回给客户端，改为显示统一的友好错误页面，防止信息泄露辅助攻击者进行漏洞探测。	后果范围控制（减少信息暴露）	不能防御注入漏洞本身；攻击者仍可通过盲注（布尔/时间差异）获取信息，只是增加了难度。
最小权	为应用程序连接数据库使用的	后果范围	不能防止数据读取泄露（如果业

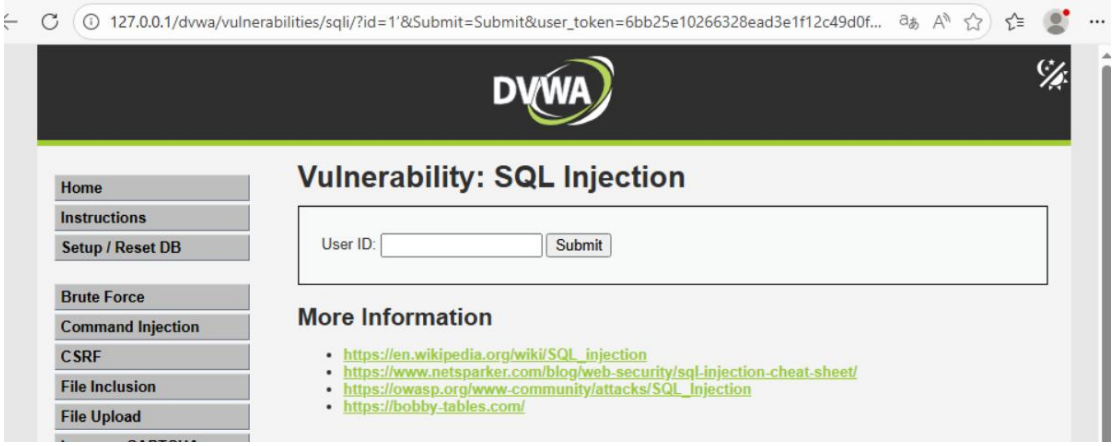
措施	解决什么问题	控制层	不能替代什么
限原则	账户仅授予必要的权限（例如只读账户不能执行 DROP、INSERT、UPDATE，且只能访问特定表），即使注入成功，攻击者也无法执行破坏性操作或窃取非授权数据。	控制（限制攻击影响）	务需要读取敏感数据，注入后仍可读取）；不能替代参数化查询（数据仍可能被批量读出）。
WAF/Web 应用防火墙（补充）	在 HTTP 请求层面检测并拦截常见的 SQL 注入攻击载荷（如 ' OR 1=1、UNION SELECT 等），作为额外防护层。	暴露面控制（部分根因缓解）	不能替代代码层的参数化查询（WAF 可被绕过或误报）；对变形 payload 或编码绕过可能失效。
定期代码审计与安全测试（补充）	通过人工或自动化工具检测代码中是否存在 SQL 拼接等危险写法，提前发现并修复漏洞。	根因发现（预防性）	不能替代运行时防护；发现漏洞后仍需修改代码（如改用参数化查询）。



impossible 级别 view_source，里面用了 $db->prepare(...) 和 bindParam



提交后页面清空且无任何错误信息：



在 DVWA 中，切换到 Impossible 级别后，输入 1' 或非法值，页面不会再显示 MySQL 原始错误，而是不显示内容。

phpMyAdmin 用户账户列表

（防护相关：Impossible 级别参数化查询代码 / 自定义错误页面 / 数据库账户权限）

分析：上图为当前数据库环境中的用户账户权限列表。其中，Web 应用程序实际使用的账户 dvwa@localhost 仅被授予了业务必需的 SELECT、INSERT、UPDATE、DELETE 权限，符合最小权限原则。

如果应用程序使用 root 账户（拥有 ALL PRIVILEGES），一旦发生注入，攻击者可执行任意操作。而当前 dvwa 账户权限受限，有效降低了风险。

最小权限原则属于后果范围控制：它不能防止注入发生，但能有效限制攻击者的破坏范围和数据泄露程度。

分层总结：

(1) 根因控制：参数化查询 —— 从设计上彻底消除“结构被污染”的可能性。

(2) 暴露面控制：输入校验、WAF —— 减少攻击者可触及的漏洞入口，但不能根治。

(3) 后果范围控制：错误处理、最小权限 —— 即使漏洞被利用，限制信息泄露和破坏范围。

核心观点：没有单一措施能完全防御 SQL 注入。参数化查询是基础，必须放在首位；输入校验提供纵深；错误处理和最小权限作为兜底，降低事故影响。

收口项 1：SQL 注入因果链/防护关系图

采用“表格 + 因果箭头”描述关系：

阶段	具体行为	是否受攻击影响	防护控制点
输入点	用户输入 1' OR '1'='1	攻击者在此构造恶意载荷	输入校验（白名单）可拦截部分，但不是根本
查询构造	后端直接拼接："SELECT ... WHERE id = '" + input + "'"	关键污染点	参数化查询在此层分离结构与数据
数据库解析/执行	SQL 解析器将输入内容当作代码，改变 where 语义	结构被污染 → 执行非预期逻辑	参数化查询后，输入不会作为代码解析
结果差异/行为证据	返回所有用户记录（普通注入）或真假状态（盲注）	证据产生	错误处理可隐藏细节；最小权限可限制危害
根因判断	结构被污染（输入进入查询结	-	-

阶段	具体行为	是否受攻击影响	防护控制点
	构，而非仅作为数据）		

因果链：

输入点（恶意 payload）→ 查询构造（直接拼接）→ 数据库解析（结构改变）→ 执行（非预期）→ 行为差异（多行回显/状态变化）

防护插入点：在“查询构造”前执行参数化查询（根因控制），在“数据库执行”后搭配最小权限 + 错误处理（后果缩减）。

收口项 2：学生自评表

1.我本章最清楚的一点是什么？

SQL 注入的核心根源是查询结构被污染，而非特殊字符本身危险。

2.我最容易混淆的一点是什么？

盲注的判断逻辑与不同盲注类型的区分方式。

3.我本次作业中哪一段最需要教师复核？

Low 与 Medium 防护机制对比、各防护措施的层级划分。

4.如果重做一次，我会优先改哪一部分？

补充盲注相关实验现象记录，完善对比分析。

收口项 3：AI 输出审核与人工复核记录

1. 本作业哪些内容使用过 AI 辅助

文本框架整理、语句润色、表格排版。

2. 哪些判断是你自己复核后保留的

实验现象、DVWA 操作流程、核心原理结论。

3. 哪些内容你发现 AI 输出不准确并进行了修正

修正 AI 将“特殊字符”作为注入根源的错误，统一为结构污染；修正防护层级错误。

4.你最终如何确认结论和证据是一一对应的

所有结论均与 DVWA 实际实验现象对应，确保原理、现象、证据一致。

提交前自查清单

我是否只在授权环境中完成实验 ✅

我的截图是否已脱敏 ✅

我是否说明了“正常查询变化”和“查询逻辑被改变”的区别 ✅

我是否把根因写成“结构被污染”，而不是只写“特殊字符”✅

我是否区分了普通 SQL 注入与盲注的证据差异 ✅

我是否把参数化查询、错误处理、最小权限放回不同控制层 ✅

我是否完成了关系图 / 关系表 ✅

我是否填写了 AI 输出审核与人工复核记录 ✅